OBASEKI, IMISIOLUWA HAIKEYS

Full-Stack Blockchain Engineer

imisioluwa@obaseki.dev | +234 803 800 4334 | obaseki.dev | Nigeria (UTC+1), open to remote
linkedin.com/in/imisioluwa-obaseki-537298318 | github.com/Haikeysgit

SUMMARY

Full-stack blockchain engineer who builds contracts that hold other people's money: vaults that cap what an AI agent can pay out, escrow-style pools that release funds only on an authorised verdict, refunds that work without the operator, and no admin path to user funds, backed by invariant tests and multi-pass audits. Shipped on Base, Monad and NEAR, plus the TypeScript backends that settle against them.

TECHNICAL SKILLS

Smart Contracts: Solidity, Rust (near-sdk), NEAR Chain Signatures, Foundry (forge, cast, anvil), OpenZeppelin, ERC20, ERC721 & EIP-2612 permit, unit, fuzz and invariant testing, gas budgets, security audits, deployment and source verification on Ethereum, Layer 2 EVM chains, Monad and NEAR

Languages: Solidity, Rust, TypeScript, JavaScript, Python, SQL

Backend: Node.js, Bun, Next.js, NestJS, FastAPI, GraphQL, REST APIs, WebSockets, PostgreSQL, viem, SIWE wallet auth, Vercel

Frontend: React, Next.js, Angular, Tailwind CSS

SMART CONTRACT PROJECTS

Bulkhead | Smart Contract Engineer, team of 2 | NEAR Hackathon

- Wrote the NEAR vault contract in Rust for Bulkhead, a firewall and contract-enforced spending cap for AI agents that hold money. The contract controls its Bitcoin and Base USDC through NEAR Chain Signatures, so no person holds a private key.
- Each tier enforces a per-payout cap, a cooldown and a round limit on-chain, so a tricked agent can release only one small fixed amount. The contract builds each transaction itself (fixed chain, token and calldata); owner actions are two-step proposals and the contract is not upgradeable.
- Built a one-command TypeScript deploy tool that creates a vault, derives each tier's Bitcoin address from the signer and registers the agent; 83 Rust unit tests and 15 TypeScript tests.
- Bitcoin testnet4 payout signed by the contract: `a8ecea7c06b387f86349322cf025233c3f170e69a2a31f27e274e70ae89e8b1a`

Stack: Rust, near-sdk, NEAR Chain Signatures, Bitcoin, Base, TypeScript, Bun

Chog Buddy | Smart Contract & Backend Engineer, team of 2 | Chogathon 2026

- Built the on-chain and backend side of Chog Buddy, a desktop pet for Chog Genesis NFT holders: SIWE wallet sign-in with NFT ownership checks on Monad mainnet, chat and diary APIs, and a daily job that writes each Chog's activity "Soul" to `ChogSoul`, stored by token ID so it moves with the NFT when sold.
- Designed Focus Bets: users stake a $CHOG test token (ERC20 + permit) that they will focus. `FocusPool` reserves each bonus at stake time and caps it at 1% of the free pot; a lost stake burns 10% and refills the pot with 90%. Self-reported focus can only grow as fast as the clock, and every bet settles exactly once.
- Ran 4 AI-assisted security audit passes and a backend security scan and fixed every finding; 54 Foundry tests (incl. invariant and gas-budget tests) and 117 backend tests.
- Live and verified on Monad testnet: FocusPool `0x0ae1F74236af4767027ED5e34aFec17D97ec5007`, ChogSoul `0xf9a7E0172a31235d99e3d93Adf33B36A39279EE4`

Stack: Solidity, Foundry, OpenZeppelin, Monad, TypeScript, Next.js, viem, PostgreSQL (Neon), Vercel

Squadjar | Smart Contract Engineer, team of 2 | Monad Metropolis Hackathon

- Wrote the contracts for Squadjar, a rotating savings (ajo/esusu) app: an ERC20 token, an on-chain trust registry, and a factory that deploys one contract per group to hold its funds and pay out each round, with no withdraw function and no admin key.
- Designed a payout hold-back sized by on-chain trust tier, so early collectors leave part of what they still owe in the jar, and debt tracking that pays a missed contribution straight to the short-paid collector. A permissionless `settleRound`, called every minute by a background service I set up, closes each round.
- 70 unit tests and 3 fuzz invariant tests (1,000 random sequences each) check that the jar's balance always matches its accounting and total debt always equals total credit; all 73 pass.
- Live and verified on Monad testnet: SquadFactory `0x7bBADfC407b7eC8941B7A72A4820Ae946dF48Ef2`

Stack: Solidity, Foundry, OpenZeppelin, Monad

ERC721 NFT Contract with On-Chain Minting | Base

- Wrote an ERC721 contract from scratch with on-chain minting and token metadata for a multi-item collection. Live on Base, indexed and tradeable on OpenSea: `0xe54425d73117727d5A858f9433612593832e1C27`

Stack: Solidity, Foundry, OpenZeppelin, Base

PROFESSIONAL EXPERIENCE

Solo Full-Stack Developer | Tektradoor *tektradoor.com* *2025 – Present*

- Architected an automated crypto futures trading engine in Python, run by AI agents, that handles the full trade lifecycle with no manual intervention.
- Built a multi-strategy technical-analysis stack gated by XGhost, a proprietary XGBoost model that scores each setup before execution, plus a re-validation engine that re-checks pending orders every 30 seconds.

Stack: Python, FastAPI, PostgreSQL, XGBoost, WebSockets

EDUCATION

B.Eng. Computer Engineering | Elizade University *Nov 2020 – Sep 2025*

Ilara-Mokin, Ondo State, Nigeria • CGPA: 4.02 / 5.00

CERTIFICATIONS & OPEN SOURCE

- Cyfrin Updraft: Blockchain Basics (2025), Solidity 101 (2025) • Udemy: Python Crash Course (2021)
- Aden HQ, Hive (AI agent framework): contributed Windows 11 compatibility fixes and onboarding documentation.